PORTFOLIO PROJECT · RETRIEVAL-AUGMENTED GENERATION (RAG)

PM Classes Study Assistant

How the whole system works

A chat assistant that answers questions from a full product-management course: 32 recorded lectures plus every PDF, Doc and Sheet shared in class. Each answer cites the exact notes page or lecture moment it came from, is fact-checked against the class material, and shows the matching slide.

32 classes & workshops covered	**66.6 h** of lecture video transcribed	**9,250** searchable pieces of class material	**60** notes files (PDF, Docs, Sheets)
99.3% of facts correct on a 183-question test	**≈ ₹3.5** cost per question (≈ $0.036)	**~12 s** until the first word appears	**≈ ₹0** hosting when nobody is using it

1 What it does
2 The big picture
3 Building the library: notes
4 Building the library: lecture videos
5 Answering a question, step by step
6 Hybrid search and RRF
7 The fact-check
8 Architecture and deployment
9 How I test it (evals)
10 What it costs
11 Decisions and trade-offs
12 Glossary

1 · WHAT IT DOES

A study partner that has sat through every class

The course material is spread across dozens of Drive folders and 66 hours of recordings. Finding "where did sir explain the Kano model?" meant scrubbing through videos. This assistant answers in seconds and shows where the answer came from.

What a student can ask

💬 Any question about the course

"What is the difference between an MVP and a prototype?" The answer cites sources inline as [1], [2], each one a notes page or a lecture moment.

📍 Where was it taught?

"Where was North Star metric covered?" Returns clickable links that open the lecture video at the exact second, plus the notes page.

📝 Class and topic summaries

"Summarise class 12" or "summarise stakeholder management" across every class that touched it. There's also a whole-course overview.

🎯 Quizzes

"Quiz me on GTM." Multiple-choice questions built from the class material and graded, with a link to where each answer was taught.

🔁 Restyle the last answer

"Make it simpler", "in Hinglish", "explain like I'm 12", "in 3 points". The same answer is rewritten, with no new search.

🖼️ Slides, 🎙️ voice, ✅ trust

Matching class slides appear under the answer. A Play button reads the answer aloud in Indian English. Every answer carries a "Fact-checked" note.

Ground rules the assistant follows

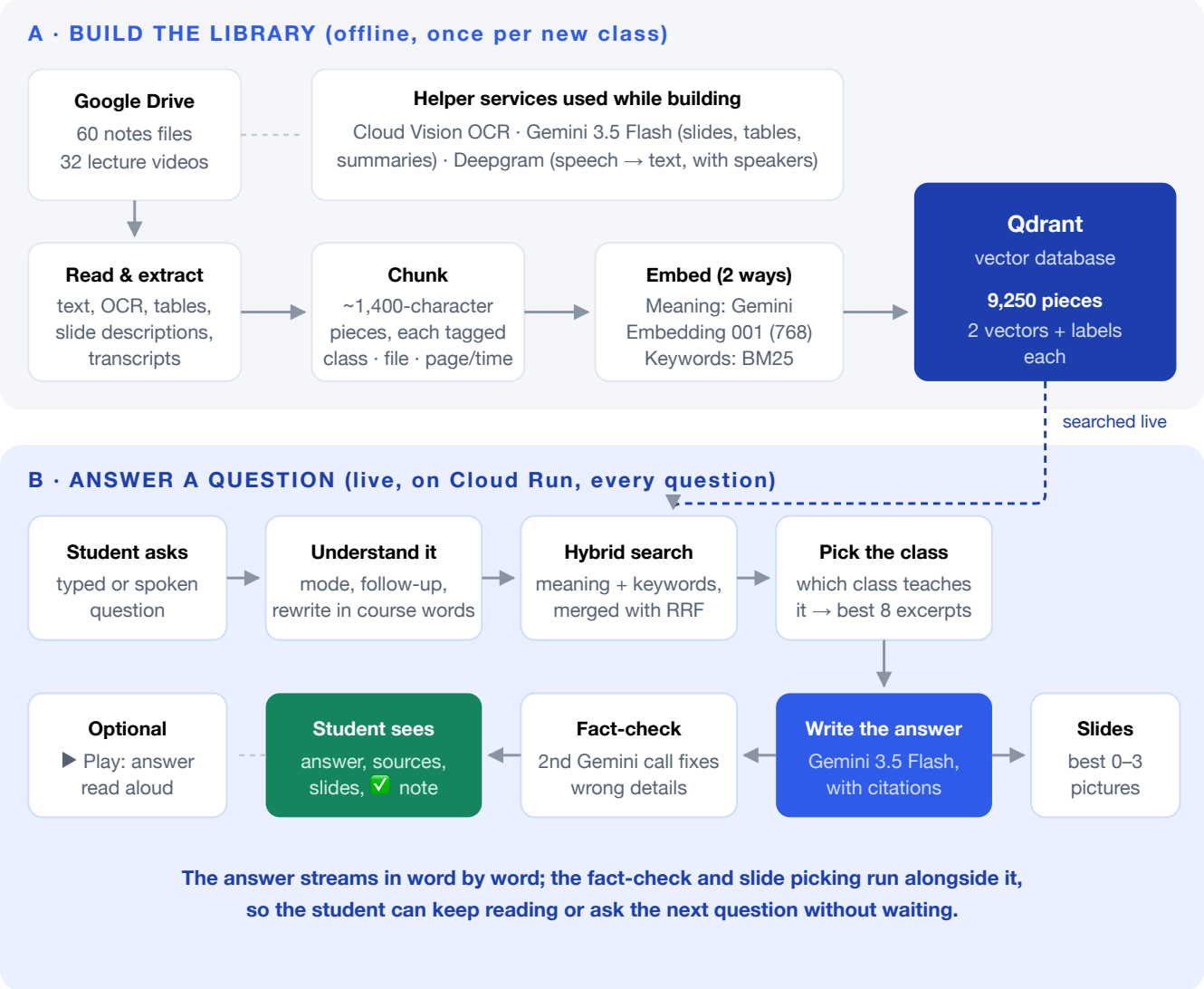
RULE	WHY IT MATTERS
Class material first	Everything presented as "from class" must appear in the retrieved excerpts. Outside knowledge only goes under a heading "*General answer (not from class)*".
Always cite	Every claim points to an excerpt, so a student can check it in one click.
Say when it wasn't covered	If nothing in class matches, it says "*This wasn't covered in the class material I have*" instead of guessing.
Teacher vs student	A student's opinion in a lecture is never presented as the teacher's teaching.
Flag slide slips	If a class slide has a clear factual slip, the assistant keeps what class showed and adds a short "⚠️ Note (general knowledge)" correction.
Low creativity	Temperature 0.2, so answers are consistent and factual, not inventive.

> **What "RAG" means here:** the AI model doesn't "know" the course. For every question the system first **retrieves** the most relevant pieces of class material, then the model writes an answer **using only those pieces**. That makes answers checkable and keeps them up to date when new classes are added.

2 · THE BIG PICTURE

Two pipelines: build the library once, then answer live

The slow, expensive work (reading PDFs, transcribing videos) happens once per new class, on my laptop. The live app only searches and writes.



Model routing: the right model for each job

JOB	MODEL	WHY THIS ONE
Rewrite the question, pick the class, score slides, grade quiz answers	Gemini 3.5 Flash-Lite	Short decisions; about 5× cheaper and fast
Write answers, fact-check, describe slides	Gemini 3.5 Flash	Needs accuracy; Flash-Lite answers scored lower in tests
Backup when Flash is busy	Gemini 2.5 Flash	Keeps the app answering during rate limits
Grade the eval, check class summaries	Gemini 3.1 Pro	Strongest judge, used offline only
Question type detection, merging, picking excerpts	Plain code	Free and instant; no AI needed

3 · BUILDING THE LIBRARY: NOTES

Turning PDFs, Docs and Sheets into searchable pieces

One command (`ingest.py`) pulls new or changed files from the course Drive folder and indexes only those. Unchanged files are skipped, so re-runs are almost free.

1 **Download from Google Drive** — Drive API · free

Read-only access to the course folder (one sub-folder per class). Google Docs are exported as PDF and Sheets as Excel, so charts and pictures aren't lost. Each file is recorded with its class number and last-changed time.

2 **Read the text, OCR the rest** — Cloud Vision

The built-in text of each page is read for free (PyMuPDF). A page with under 200 characters is really a picture of a slide, so it goes to **Google Cloud Vision OCR**, as do large screenshots.

3 **Describe diagrams in words** — Gemini 3.5 Flash

Diagram and screenshot pages get a short "Visual:" description from **Gemini 3.5 Flash**, so a question about a framework diagram can find that page.

4 **Rebuild tables as real rows and columns** — PyMuPDF → Gemini

Plain PDF text jumbles tables ("Mutual funds / High / Low / Low"), so the model guesses which value sits in which column. Each table is rebuilt as a grid. When Gemini reads one off the image, **97%+ of its words must exist on the page**, so it can't invent cells.

5 **Cut out the pictures** — Cloud Storage

Slides and screenshots are saved to Cloud Storage, so the app can show them under answers.

6 **Chunk** — free

Pages are split into pieces of at most **1,400 characters** along paragraph and bullet breaks, with a **200-character overlap** so a sentence cut at the edge keeps its context.

7 **Embed and store** — Gemini Embedding

Every piece gets two "fingerprints" (see section 6) and is stored in Qdrant with its labels. OCR, descriptions and tables are cached per file version, so they're only paid for once.

What one stored piece looks like

Text that gets embedded (a header + the content)

```
Class 6 — MVP | Case Study The MVP.pdf |
page 7

A prototype validates the idea… An MVP
tests product-market fit…
```

Labels stored with it

- class number, class title, teacher
- file name, page, Drive link
- how it was read (text / OCR / visual)
- attached pictures (yes/no)
- source: notes, lecture, slide or summary

Putting the class and file name in the header helps search: a question about "Class 6" or "the MVP case study" matches the right pieces.

4 · BUILDING THE LIBRARY: LECTURE VIDEOS

Making 66 hours of class recordings searchable

Most of what's taught is said out loud, not written in the notes. Each recording becomes timed transcript pieces, slide pieces and a class summary, all linking back to the exact second in the video.

1 · Download
The .mp4 from Drive (~1.3 GB each). Deleted after processing.

→

2 · Split
ffmpeg makes small audio + a screen frame every 5 s. Only frames where the screen changed are kept.

→

3 · Transcribe
Deepgram Nova-3: Hindi + English mixed, who-spoke-when, timestamps.

→

4 · Read screen
Cloud Vision OCR on every kept frame (11,123 frames in total).

5 · Name speakers
Gemini reads the transcript + screen and labels "Speaker 0" as the teacher, the rest as students.

→

6 · Lecture pieces
~1,400 characters of speech, the text on screen at that time, the time range and a link to that second.

→

7 · Slide pieces
Gemini 3.5 Flash describes each slide, plus what was said while it was up.

→

8 · Class summary
Study notes for the whole class with timed sections, fact-checked by Gemini 3.1 Pro.

An example lecture piece

```
Class 1 — Introduction to Product Management with AI | Lecture video (Instructor B) | 19:15–20:22

[19:15] Instructor B: Let's take a pet care app. The parent enters the date of birth, breed, weight…
[19:48] Instructor B: Earlier, the hard part was the database. It took 8 months to a year to build…

[On screen] Product Intelligence Layer · Foundation model · Query → Response
```

Because each piece carries its time range, an answer can say "In Class 1 at 19:15, Instructor B explained…" and link straight to that moment.

What's in the library now

KIND OF PIECE	WHAT IT IS	COUNT
Notes	Pages from PDFs, Docs and Sheets shared in class	2,176
Lecture	Timed transcript chunks with on-screen text	2,580
Slide	One slide shown in a video: description, text and what was said	4,185
Summary	Class summary sections (32 classes)	309
Total	**Across 32 classes and workshops, 2 teachers**	**9,250**

Fixing transcription slips: speech-to-text mishears names ("Namayatri" for Namma Yatri, "Moscow" for MoSCoW). A fix-list corrects known terms, and the answer prompt quietly fixes obvious ones. Text read off slides is preferred for spelling names.

Notes and lectures take turns: there are many short lecture pieces, so they could crowd the notes out of results. Search runs on notes and lectures separately and alternates them.

5 · ANSWERING A QUESTION, STEP BY STEP

What happens in the ~12 seconds after you press Enter

Each step uses the cheapest tool that does the job well. Plain code rules are free, Flash-Lite handles quick decisions, and Flash does the writing and checking.

0 — What kind of request is it? `code · free`

Word rules spot quizzes, class summaries, topic summaries, the course overview and "make it simpler / shorter" requests. Restyle requests skip search and rewrite the last answer.

1 — Handle follow-ups `code · free`

"Explain the second one" has no topic, so earlier questions are added to it for searching. "What is Figma?" names its own topic, so it's searched alone and old topics don't leak in.

2 — Rewrite in course words `Flash-Lite`

Students say "how are we graded", notes say "evaluation criteria". Flash-Lite rewrites the question in course language, and **both versions** are searched.

3 — Hybrid search `Embedding` `Qdrant`

The question is embedded, then Qdrant runs meaning search + keyword search (top 30 each), merged with **RRF**, for notes and lectures separately. A wide net: 40 pieces per search.

4 — Merge the two searches `code · free`

Pieces that rank high in both searches move to the top (score = sum of 1 ÷ (rank + 5)).

5 — Find the class that teaches the topic `Flash-Lite`

"Matrix" appears in many classes. Each class gets a "heat" score from its best matches, and Flash-Lite reads the top 5 classes' passages to decide which **teach** the topic and which only mention the word.

6 — Choose the final 8 excerpts `code · free`

Most slots go to the teaching class(es), including the teacher explaining it in the lecture. For "where was it taught?" questions, a few extra lecture moments are added.

7 — Write the answer `Gemini 3.5 Flash`

Gemini 3.5 Flash gets the rules, the last 6 chat messages and the numbered excerpts. It streams an answer with [n] citations and video links. If Google is busy, it falls back to Gemini 2.5 Flash.

8 — Pick slides (at the same time) `Flash-Lite`

15 candidate slides from the teaching classes are scored 0–10 for fit by Flash-Lite. Only scores of 7 or more are shown, at most 3. No picture is better than a wrong one.

9 — Fact-check (in the background) `Gemini 3.5 Flash`

A second Flash call checks every number, name, table cell, class number and lecture time against the excerpts and swaps in corrections. See section 7.

10 — Read aloud (only if Play is pressed) `Flash-Lite TTS`

Gemini Flash-Lite text-to-speech, "Kore" voice, Indian English. Saved, so a second play is free.

> **Timing:** the first word appears after about 12 seconds and the full answer at about 16. The fact-check takes about 4–5 more seconds, but it runs in the background, so the student can already read or ask the next question.

6 · HYBRID SEARCH AND RRF

Two ways to search, combined into one list

Each piece of class material is stored with two "fingerprints". Searching with both catches what either one alone would miss.

🧠 Meaning search (dense vectors)

Gemini Embedding 001 turns text into 768 numbers that capture its **meaning**. Similar ideas land close together, even with different words.

Good at: "how do I decide what to build first?" → finds *prioritisation frameworks*.
Weak at: exact names, acronyms, numbers ("RICE", "PhonePe", "LTV").

🔤 Keyword search (BM25, sparse vectors)

The classic search-engine method. It scores pieces that contain the question's **exact words**, giving more weight to rare words.

Good at: "RICE framework", "Namma Yatri", "Kano".
Weak at: synonyms and paraphrases ("graded" vs "evaluation criteria").

How RRF merges the two lists

Reciprocal Rank Fusion ignores the raw scores (which are on different scales) and looks only at **rank**. Each piece gets `1 ÷ (k + rank)` from every list it appears in, and the totals are added up. A piece ranked high in both lists wins.

Meaning search	Keyword search	After RRF (final order)
#1 Prioritisation overview	#1 RICE scoring slide	**1. RICE scoring slide**
#2 RICE scoring slide	#2 RICE worked example	2. Prioritisation overview
#3 Roadmap planning	#3 Prioritisation overview	3. RICE worked example

Question: "How does the RICE framework work?"
The RICE slide is #2 + #1, so it scores highest overall. The overview (#1 + #3) comes next.
Neither list alone had this order. That's why combining them helps.

Are we using a re-ranker? No, and here's why

	RRF (WHAT WE USE)	RE-RANKER (NOT USED)
What it does	Merges lists by rank, using simple maths	A second AI model re-reads every candidate next to the question and re-scores it
Cost	Free: runs inside Qdrant	An extra paid model call on every question
Speed	Milliseconds	Adds roughly 0.5–2 seconds
Accuracy	Good	Usually better at ordering

> A re-ranker normally runs *after* RRF, so they aren't rivals. This system already gets most of a re-ranker's benefit from two cheaper steps: **the class-routing step** (Flash-Lite judges which classes really teach the topic) and the **fact-check**, which catches what slips through. With 99.3% of facts correct, a re-ranker is only worth adding if evals start showing wrong passages being picked.

7 · THE FACT-CHECK

A second pair of eyes on every answer

This came from a real failure: an answer read a High/Low value from the wrong column of a table in the notes. The fix was two-fold. Tables are now stored as real grids (section 3), and every answer is checked before the student relies on it.

1 · Answer is written
Shown to the student straight away. Fact-check starts in the background.

→

2 · Checker reads
The answer + the same excerpts it was written from (with links and times).

→

3 · Lists problems
Each one as JSON: the exact wrong text, the corrected text, and why.

→

4 · Swaps fixes in
The answer updates on screen, with a note under it.

What it checks

- **Numbers**: percentages, prices, counts
- **Table values**: the right row *and* the right column
- **Names**: companies, products, people, frameworks and their steps
- **Examples**: which example illustrates which point
- **Who said it**: teacher vs student, and the class number
- **Lecture times**: "Class 18 at 1:16:27" must fall inside a cited excerpt's time range
- **Citations**: [3] must point to an excerpt that supports the sentence
- **Files and page numbers**

What it leaves alone

- Wording, style and fair paraphrases
- The "General answer (not from class)" section
- "⚠️ Note (general knowledge)" corrections
- Correct names that the *transcript* misspelled. It never swaps "MoSCoW" back to the misheard "Moscow"

Safety net

If the fixes would remove more than 60% of the answer, the excerpts were probably the wrong ones, not the answer. The original is kept with a warning: "⚠️ *Couldn't confirm most of this. Double-check it.*"

Example of a correction

Before
"…mutual funds have Low conversion [7]."

After
"…mutual funds have High conversion [7]."
Why: Notes table, row "Mutual funds", column "Conversion" = High [7]

The student sees "✅ Fact-checked: corrected 1 detail against the class material" and can expand it to see exactly what changed.

Tuning it: medium vs low "thinking"

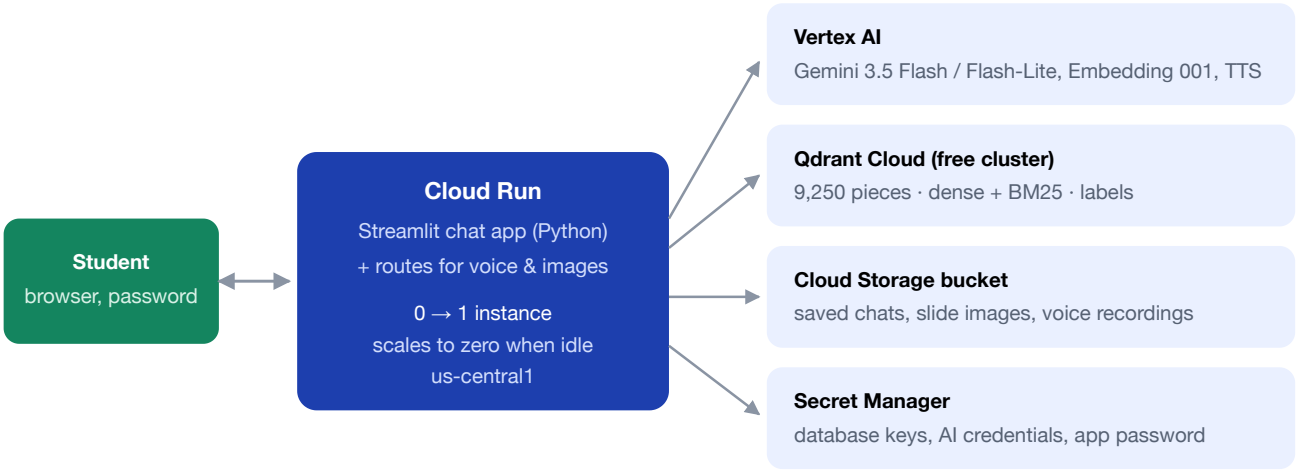
SETTING (50 TEST QUESTIONS)	FACTS CORRECT	CHECK TIME	COST PER ANSWER
Fact-check on **medium** thinking (before)	95.9%	10.2 s	₹4.9
Fact-check on **low** thinking (live since 26 Oct 2026)	95.6%	4.3 s	₹3.4
Flash-Lite as the checker (rejected)	93.5%	–	₹0.7

Low thinking gave the same accuracy (within noise), finished 2.4× faster and cost 30% less. Flash-Lite was far cheaper but returned empty checks in 55 of 56 cases, which meant no real checking, so it was not adopted.

8 · ARCHITECTURE AND DEPLOYMENT

Where everything runs

Built almost entirely on Google Cloud, with a free Qdrant cluster for search. The app scales to zero when nobody is using it.



Offline, on my Mac:
ingest.py → Google Drive, Cloud Vision OCR, Deepgram, Gemini → writes into Qdrant + Cloud Storage

How code goes live

1 · Test locally
Run the app on my Mac and the eval test set.

→

2 · Commit & push
To the *main* branch on GitHub (private).

→

3 · Build
Cloud Build makes a Docker image. The build fails if the search model can't load.

→

4 · Store
The image is saved in Artifact Registry, tagged with the commit ID.

→

5 · Deploy
Cloud Run swaps to the new version. Rollback = redeploy the old image.

Tech stack

LAYER	CHOICE
Interface	Streamlit (Python), chat history, sidebar class filter, voice input, dark/light theme
Language models	Gemini 3.5 Flash, 3.5 Flash-Lite, 2.5 Flash (backup), 3.1 Pro (eval only), on Vertex AI
Embeddings	Gemini Embedding 001, 768 dimensions (meaning) + BM25 via FastEmbed (keywords)
Vector database	Qdrant Cloud: hybrid search, RRF, label filters (class, teacher, source)
Document reading	PyMuPDF, Google Cloud Vision OCR, Gemini vision for diagrams and tables
Speech	Deepgram Nova-3 (lectures, Hindi + English); Gemini for dictation and read-aloud
Hosting & ops	Cloud Run, Cloud Build, Artifact Registry, Cloud Storage, Secret Manager, Docker, uv

9 · HOW I TEST IT (EVALS)

Measured accuracy, not promises

Before any change goes live, a test set is run through the **exact same steps the app uses**. Every answer is broken into individual facts, and each fact is graded against the class material.

183
test questions across 13 types

3,748
individual facts graded

99.3%
facts correct (3,721 of 3,748)

168/183
answers with zero mistakes

Four kinds of check

1 · AI grader, fact by fact

Gemini 3.1 Pro (a stronger model than the one answering) splits each answer into claims and marks each correct, wrong or unsupported by the excerpts. It also checks that the must-have facts are covered.

2 · Exact rule checks

No AI judgement involved: lecture times must fall inside real excerpt ranges, required words must appear, banned words must not, and "not covered" questions must say so.

3 · Search quality

For cases with hand-picked evidence: did search find the right pieces (recall), and how many of the 8 were useful (precision)?

4 · Before/after cost tests

A fixed 50-question set records cost, tokens and time per step, so a cheaper setting is only adopted if accuracy holds.

Results by question type (full run, 25 Oct 2026)

TYPE	EXAMPLE	CASES	FACTS CORRECT
Facts	"What is the difference between MVP and prototype?"	71	99.6%
Class summaries	"Summarise class 12"	32	99.6%
Numbers	"How many users should a usability test have?"	20	100%
Tables	Values read from a High/Low grid in the notes	12	99.4%
Follow-ups	"…and how is it different from the second one?"	12	92.3%
Where taught	"Where was North Star metric covered?"	10	100%
Topic summaries	"Summarise everything on pricing"	6	99.6%
Quiz, pictures, slides	Quiz keys, slide matches	14	100%
Not covered, language, filter	Off-syllabus questions, Hinglish, one class only	6	100%

Caveats: follow-ups are the weakest type (92.3%). On a harder 50-question cost test set, accuracy is 95.6–96.4%. The AI grader is sometimes wrong itself (for example, it can trust jumbled slide text), so its "wrong" verdicts are checked against the transcript before anything is "fixed".

Running it: a full run takes about 40 minutes and costs about $22 (₹2,100), mostly for the Pro grader. Smaller targeted runs (one question type, or the 50-question set) cost $1–7.

10 · WHAT IT COSTS

Cost in every direction

All prices are Google's / vendors' list prices (Oct 2026) at ₹94.6 = $1. Per-question numbers are **measured** on 56 test questions. Ingest numbers are **estimates** from file counts and list prices.

① Cost per question (measured)

STEP	WHAT'S PAID FOR	$	₹
Understand, search, route, write	Flash-Lite rewrite + routing, embedding the question, then Flash reads ~7,600 tokens and writes ~1,460 + ~710 "thinking" tokens	0.0206	1.95
Fact-check	Flash reads ~5,100 tokens, writes ~800 + ~780 thinking tokens	0.0149	1.41
Slide picking	Flash-Lite scores 15 candidate slides	~0.0015	0.10–0.20
Per question	**Simple questions are cheaper, long topic summaries cost more**	**≈ 0.037**	**≈ 3.5** (₹2–10)
Read aloud (optional)	Only when ▶ Play is pressed; replaying is free	0.01–0.04	1–4

Why output matters most: Flash charges $1.50 per million tokens read but $9.00 per million written (6× more), and "thinking" counts as written.

② One 10-question study session

10 answers × ₹3.5	₹35
Slides for each	₹1.5
3 answers read aloud	₹6
Session total	**≈ ₹40–45 ($0.45)**

Follow-ups cost about the same as new questions, because only the last 6 messages are sent. The course overview is free after the first time (it's cached).

③ Running costs per month

Cloud Run (scales to zero)	≈ ₹0 idle
Qdrant Cloud free cluster	₹0
Storage, registry, secrets	a few ₹
Fixed monthly	**≈ ₹0–100**

Trade-off: the first visit after a quiet spell waits tens of seconds while the app starts. Keeping one copy always running would cost about $13–20 (₹1,200–1,900) a month.

Where the money goes in one question



The green sliver is slide picking (~5%). Search itself (Qdrant, RRF) is free. Almost all cost is Gemini reading excerpts and writing text.

Adding new material: a one-time cost

Building the library costs far more than answering, but it's paid once per class. Everything is cached, so re-running is nearly free. These are **estimates** from file counts and list prices; the Deepgram rate is approximate.

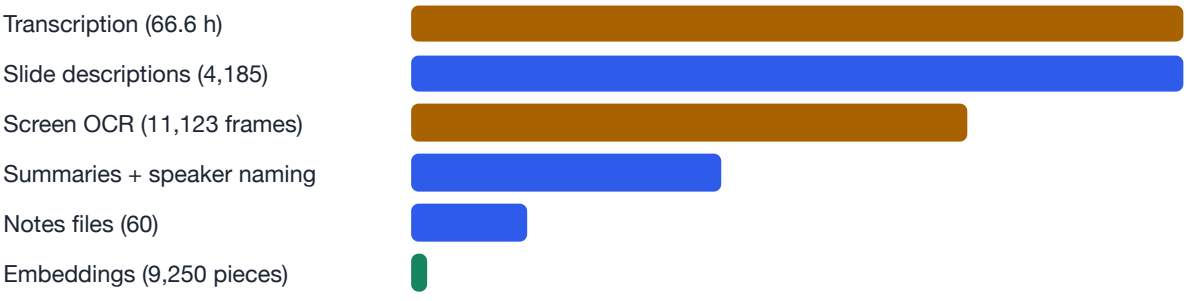
④ One new 2-hour lecture video

STEP	BASIS	₹
Transcription (Deepgram Nova-3, multilingual + speakers)	~120 min × ~$0.0052/min	≈ 60
Screen OCR (Cloud Vision)	~350 changed frames × $1.50 per 1,000 (first 1,000 a month free)	0–50
Slide descriptions (Gemini 3.5 Flash)	~130 slides × ~$0.005	≈ 60
Speaker naming, class summary, Pro check of the summary	a few long Gemini calls	≈ 25
Embeddings (~290 pieces)	$0.15 per million tokens	≈ 2
Per lecture video		**≈ ₹150–200**

⑤ One new notes file

FILE	WHAT COSTS MONEY	₹
Text-only PDF or Doc (~20 pages)	Embeddings only	< 1
Slide deck with images and diagrams (~20 pages)	OCR for image pages, Gemini for diagrams and tables, embeddings	≈ 2–10

⑥ The whole course, built once (estimate)



Total, one time	**≈ ₹6,500**

Orange = outside services (Deepgram, Cloud Vision), blue = Gemini, green = embeddings. Turning meaning into numbers is the cheapest part. Reading video, both speech and screen, is the most expensive.

> **Testing costs too:** a full 183-question eval run costs about ₹2,100 ($22), mostly for the Gemini 3.1 Pro grader. The 50-question cost set costs ₹300–650 per run.

How cost was cut, and what's next

Rule: a cheaper setting is only adopted if the eval shows accuracy holds. Every idea below was tested on the same 50-question set.

Already done

CHANGE	EFFECT	SAVING
Fact-check on "low" thinking instead of "medium"	Same accuracy (95.6% vs 95.9%), check 10 s → 4 s	₹4.9 → ₹3.4 per answer (–30%)
Flash-Lite voice for read-aloud	Sounded as good in a side-by-side listen	~40% cheaper per play
Cheap model for small decisions	Rewrite, routing, slide scoring and quiz grading run on Flash-Lite	~5× cheaper per call
Caching	Course overview, voice recordings and every ingest step are saved and reused	repeat = ₹0
No re-ranker	RRF is free; class routing + fact-check cover the gap	avoids an extra call per question
Scale to zero	No always-on server	≈ ₹1,200–1,900/month avoided

Tested and rejected

IDEA	RESULT	DECISION
Flash-Lite for answers *and* fact-check	₹0.66 per question, but 93.5% accurate; only 78% on "where was it taught"; the checker returned empty output	Rejected: too many errors
Share excerpts between answer and check to get cache discounts	Google gave no cache hits between the two calls	Dropped

Next options (not yet live)

IDEA	ESTIMATED EFFECT	RISK
Route easy questions to Flash-Lite, keep Flash for numbers and "where" questions, keep the Flash fact-check	≈ ₹2.6 per question (–25%). Built and switched off; first test 94.8% vs 96.1%	Small accuracy drop
Skip the fact-check on "make it simpler" rewrites	Saves ₹1.4 on those turns	Rewrites can still change facts
Skip slide picking for small talk, cache slide scores	Saves ₹0.1–0.2 per turn	Low

Bottom line: about **₹3.5 a question** and **₹40–45 for a 10-question study session**, with near-zero fixed cost. Adding a new 2-hour class costs about **₹150–200**, once.

11 · DECISIONS AND TRADE-OFFS

The choices that shaped the system

The interesting part of a RAG system is less "which model" and more how you decide. These are the main calls I made and why.

Hybrid search over meaning-only search

PM classes are full of names and acronyms (RICE, MoSCoW, PhonePe). Meaning search misses them, and BM25 catches them. Combining both with RRF costs nothing extra.

Class routing instead of a re-ranker

All PM material looks alike to an embedding model (0.8–0.93 similarity between any two classes). The real problem was *which class* teaches a topic, so a cheap Flash-Lite call answers exactly that.

Fact-check over a bigger answer model

A second, focused check catches specific slips (table cells, times) more cheaply than paying for a larger model on every answer. It runs in the background, so it adds no waiting.

Tables as grids

Rather than asking the model to "be careful", the input was fixed: tables rebuilt as rows and columns, with a guard so Gemini can't invent cells.

Lectures as first-class sources

Transcribing 66 hours cost more upfront, but most teaching is spoken. Timed links let students jump to the exact moment, which the notes alone can't do.

Evals before every change

Each prompt, search or cost change runs against the test set. That's how "low thinking" was proven safe and "Flash-Lite everywhere" was proven unsafe.

Scale to zero

For a class-sized audience, a slow first load after idle is an acceptable price for near-₹0 hosting. I chose not to keep a copy running all the time.

Honest "not covered"

The assistant must say when class didn't cover something, and label outside knowledge separately. Students should trust that "from class" really means from class.

What I'd improve next

- **Follow-up questions** are the weakest area (92.3%). Better tracking of what "it" refers to would help most.
- **Search precision** is about 55%: roughly half of the 8 excerpts are directly useful. A re-ranker or tighter routing could lift this, and the eval would show whether it's worth the cost.
- **Auto-sync**: run the library update on a schedule instead of by hand when a new class is uploaded.

> **In one sentence:** the system reads every class note and recording once, stores them as searchable pieces with two kinds of fingerprint, and for each question finds the right class, writes a cited answer with Gemini and fact-checks it, all for about ₹3.5.

12 · GLOSSARY

Terms in plain English

TERM	MEANING
RAG	Retrieval-Augmented Generation. Find relevant material first, then have the AI answer using only that material.
Chunk / piece	A short section of class material (up to ~1,400 characters) that can be searched and quoted on its own.
Embedding	A list of numbers (here 768) representing what a piece of text *means*. Texts with similar meaning get similar numbers.
Dense vs sparse vector	Dense = the embedding (meaning). Sparse = a list of which words appear and how important they are (keywords).
BM25	A classic keyword-scoring formula used by search engines. Rare words that match count more.
Vector database	A database built to find the stored vectors closest to a question's vector, fast. Here: Qdrant.
Hybrid search	Running meaning search and keyword search together and combining the results.
RRF	Reciprocal Rank Fusion. Combines several ranked lists by adding 1 ÷ (k + rank) for each item. Free and simple.
Re-ranker	An extra AI model that re-reads the top results and re-orders them. More accurate, but adds cost and delay. Not used here.
Model routing	Sending each job to the cheapest model that does it well (Flash-Lite for decisions, Flash for writing).
Token	The unit AI models read and write and are billed by. Roughly ¾ of an English word.
Thinking tokens	Hidden reasoning the model does before answering. Billed as output, so "low thinking" saves money.
Temperature	How creative or random the model is. 0.2 = steady and factual.
OCR	Optical Character Recognition. Reading text out of an image, like a screenshot of a slide.
Diarization	Working out who spoke when in a recording (teacher vs students).
Grounding / citations	Tying each claim to a specific excerpt [n] that the student can open and check.
Eval	A repeatable test set used to measure quality before and after a change.
Precision / recall	Precision: how many retrieved pieces were useful. Recall: how many of the useful pieces were found.
Cloud Run	Google's service that runs the app in a container and adds or removes copies with traffic, down to zero.
Cold start	The wait while a scaled-to-zero app starts up for the first visitor.